

Deep Learning for NLP

Training LLMs

Memory and compute requirements

32-bit float (FP32)

sign exponent (8 bits) significand (23 bits)

0 1 0 0 0 0 0 0 0 1 0 0 1 0 0 1 0 0 0 0 1 1 1 1 1 0 1 1 0 1 1

$$(-1)^0 \times 2^{100-127} \times 1.5707964 = 3.1415927$$

16-bit float (FP16)

sign exponent (5 bits) significand (10 bits)

0 1 0 0 0 0 1 0 0 1 0 0 0 0

$$(-1)^0 \times 2^{100-127} \times 1.571 = 3.141$$

Learning goals

- Learn about different contributions to compute requirements
- Learn how model size components influence memory requirements

NUMBER OF PARAMETERS: NOTATION

- In this slide set, we use P for the number of parameters.
- (Unfortunately, we use N for the number of parameters in other slide sets.)

COMPUTE REQUIREMENTS

COMPUTE REQUIREMENTS

Basic equation: Cost to train a transformer (decoder) model:

$$C \approx \tau T = 6PD$$

► Source: Quentin et al., 2023

COMPUTE REQUIREMENTS $C \approx \tau T = 6PD$

where:

- τ is throughput of hardware: (No. GPUs) x (FLOPs/GPU)
- T is the time spent training the model, in seconds
- P is the number of parameters in the model
- D is the dataset size (in tokens)
- C : No. of floating-point operations to train the model:
$$C = C_{forward} + C_{backward}$$
- $C_{forward} \approx 2PD$; $C_{backward} \approx 4PD$
 - $2PD$: **2** comes from the multiply-accumulate operation used in matrix multiplication
 - $4PD$: backward pass approximately twice the compute of the forward pass
 - In the backward pass at each layer, gradients have to be calculated for the weights at that layer and for the previous layers' outputs.

COMPUTE UNITS

C (actually, C per time) can be measured in different units:

- FLOPs = FLOP-seconds which is [Floating Point Ops / Second]
 - We also use multiples:
GFLOP-seconds, TFLOP-seconds etc.
 - Other multiples like PFLOP-days are used in papers
 - 1 PFLOP-day = $10^{15} \cdot 24 \cdot 3600$ FLOP-seconds
 - Actual FLOPs are always lower than the advertised theoretical FLOPs
- GPU-hours
 - GPU model is also required, since they have different compute capacities

PARAMETERS VS DATASET

- Model performance depends on number of parameters P , but also on number of training tokens D
- One proposed optimal tradeoff between P and D is: $D = 20P$
 - This was posited for Chinchilla models ► Hoffmann et al., 2022, no longer seen as valid.
- Training an LLM on less than 200 billion tokens is not recommended
- One rule of thumb: First determine the uppermost inference cost, and then train the biggest model within that boundary
- Different ways to determine P : based on available data, compute budget or inference time

MEMORY REQUIREMENTS

MEMORY REQUIREMENTS

Common questions:

- How big is this model in bytes?
- Will it fit/train on my GPUs?

Model size components:

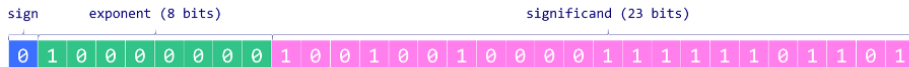
- Model parameters
- Optimizer states
- Gradients
- Activations

NUMBER REPRESENTATIONS

- Pure fp32: single precision floating point number as defined by ▶ IEEE 754 standard, takes 32 bits or 4 bytes
- fp16: half precision float number as defined by ▶ IEEE_754-2008, occupying 16 bits or 2 bytes
- bf16 or brain floating point 16, developed by Google Brain project, occupying 16 bits or 2 bytes
 - bf16 has a larger dynamic range than fp16 (8 exponent bits instead of 5)
 - bf16 is less precise, but also less likely to suffer from underflow or overflow
- INT8: integer (e.g., from -128 to 127), occupying 8 bits or 1 byte

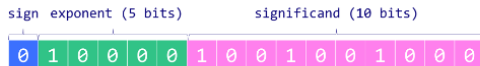
FP32/FP16 (▶ (MAXIME LABONNE))

32-bit float (FP32)



$$(-1)^0 \times 2^{128-127} \times 1.5707964 = 3.1415927$$

16-bit float (FP16)



$$(-1)^0 \times 2^{128-127} \times 1.571 = 3.141$$

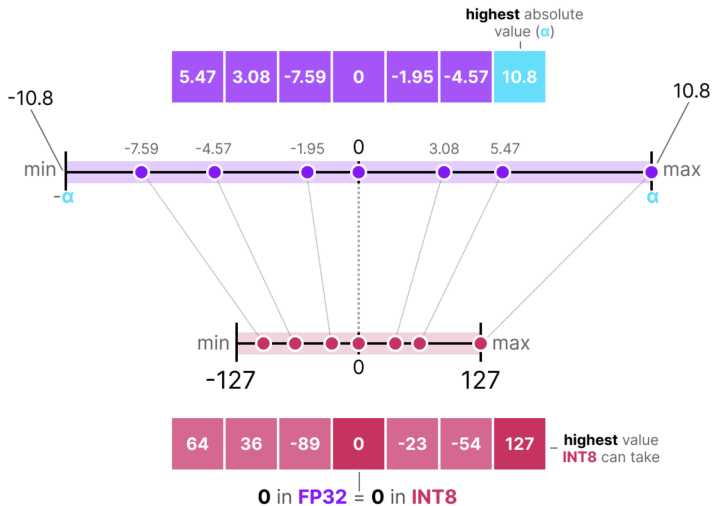
You can represent every float like this: $(-1)^S \cdot 2^{e-bias} \cdot 1.m$

INT8 QUANTIZATION

It's hard to fit meaningful floats into 8 bits, but you can use integer quantization: INT8.

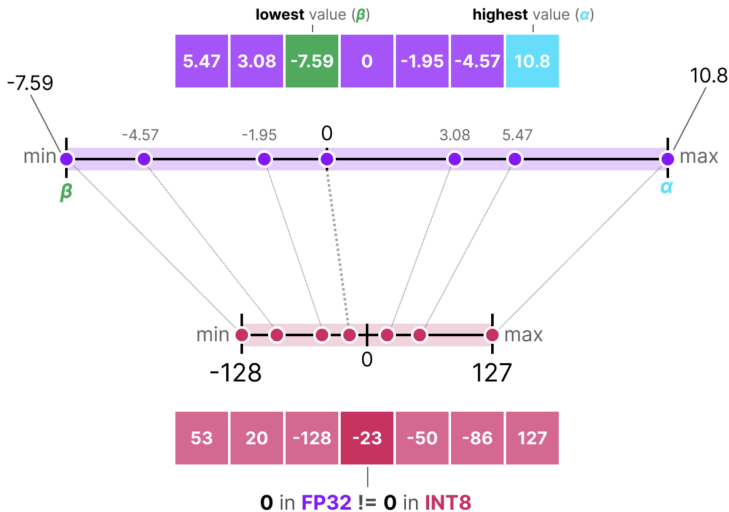
- $\text{Real_number} = \text{stored_integer} * \text{scaling_factor}$
- **Absmax quantization (symmetric):**
 - $X_{\text{quant}} = \text{round} \left(\frac{127}{\max|\mathbf{X}|} \cdot \mathbf{X} \right)$
- **Zero-point quantization (assymmetric):**
 - $\text{scale} = \frac{255}{\max(\mathbf{X}) - \min(\mathbf{X})}$
 - $\text{zeropoint} = -\text{round}(\text{scale} \cdot \min(\mathbf{X})) - 128$
 - $X_{\text{quant}} = \text{round}(\text{scale} \cdot \mathbf{X} + \text{zeropoint})$
 - (draw example)

ABSMAX INT8 (SYMMETRIC)



Link: Maarten Grootendorst

ZERO-POINT INT8 (ASYMMETRIC)



ZERO-POINT QUANTIZATION

- We assume: $\min(\mathbf{X}) \leq 0 \leq \max(\mathbf{X})$
- $\text{scale} = \frac{255}{\max(\mathbf{X}) - \min(\mathbf{X})}$
- $\text{zeropoint} = -\text{round}(\text{scale} \cdot \min(\mathbf{X})) - 128$
- $X_{\text{quant}} = \text{round}(\text{scale} \cdot \mathbf{X} + \text{zeropoint})$
- $X_{\text{quant}}(0) = \text{zeropoint}$
- $X_{\text{quant}} = \text{round}(\text{scale} \cdot \mathbf{X} + \text{zeropoint})$
 $= \text{round}(\text{scale} \cdot \mathbf{X} + -\text{round}(\text{scale} \cdot \min(\mathbf{X})) - 128)$
 $= \text{round}(\text{scale} \cdot (\mathbf{X} - \min(\mathbf{X}))) - 128$
 $= \text{round}(255 \cdot \frac{\mathbf{X} - \min(\mathbf{X})}{\max(\mathbf{X}) - \min(\mathbf{X})}) - 128$
- $X_{\text{quant}}(\max(\mathbf{X})) = 255 - 128 = 127$
- $X_{\text{quant}}(\min(\mathbf{X})) = 0 - 128 = -128$

Pretraining Large Language Models with NVFP4

NVIDIA

Abstract. Large Language Models (LLMs) today are powerful problem solvers across many domains, and they continue to get stronger as they scale in model size, training set size, and training set quality, as shown by extensive research and experimentation across the industry. Training a frontier model today requires on the order of tens to hundreds of yottaflops, which is a massive investment of time, compute, and energy. Improving pretraining efficiency is therefore essential to enable the next generation of even more capable LLMs. While 8-bit floating point (FP8) training is now widely adopted, transitioning to even narrower precision, such as 4-bit floating point (FP4), could unlock additional improvements in computational speed and resource utilization. However, quantization at this level poses challenges to training stability, convergence, and implementation, notably for large-scale models trained on long token horizons.

Link: [Nvidia paper on NVFP4](#)

MODEL PARAMETERS

Parameter size depends on chosen representation:

- Pure fp32: $Mem_{model} = 4 \text{ bytes/param} \cdot N_{params}$
- fp16 or bf16: $Mem_{model} = 2 \text{ bytes/param} \cdot N_{params}$
- INT8: $Mem_{model} = 1 \text{ byte/param} \cdot N_{params}$

It is common to use mixed representations:

- fp32 + fp16
- fp32 + bf16

OPTIMIZER STATES

AdamW: $Mem_{\text{AdamW}} = 8 \text{ bytes/param} \cdot N_{\text{params}}$

- Momentum: 4 bytes/param
- Variance: 4 bytes/param

bitsandbytes (8-bit optimizer): $Mem_{\text{optimizer}} = 2 \text{ bytes/param} \cdot N_{\text{params}}$

- Momentum: 1 byte/param
- Variance: 1 byte/param

GRADIENTS

They are usually stored in the same datatype as the model parameters.

Their memory overhead contribution is:

- fp32: $Mem_{grad} = 4 \text{ bytes/param} \cdot N_{params}$
- fp16 or bf16: $Mem_{grad} = 2 \text{ bytes/param} \cdot N_{params}$
- INT8: $Mem_{grad} = 1 \text{ byte/param} \cdot N_{params}$

ACTIVATIONS

- GPUs are bottlenecked by memory, not FLOPs
- Save GPU memory by recomputing activations of certain layers
- Various schemes for how exactly this is implemented.

Total memory when training **without** activations:

$$Mem_{training} = Mem_{params} + Mem_{opt} + Mem_{grad}$$

Total memory when training **with** activations:

$$Mem_{training} = Mem_{params} + Mem_{opt} + Mem_{grad} + Mem_{activ}$$